

ABSTRACT

SQL Injection (SQLi) attacks remain a critical threat to database-driven applications, enabling attackers to manipulate backend databases and compromise sensitive information. Traditional rule-based detection systems often struggle to adapt to the evolving landscape of attack techniques. This project presents an enhanced SQLi detection and classification system leveraging a deep learning approach with Long Short-Term Memory (LSTM) networks. The LSTM-based model effectively classifies SQL queries as malicious or benign and further categorizes detected SQLi attacks into specific types. To mitigate overfitting and improve generalization, regularization techniques are incorporated into the model's architecture. The system demonstrates high accuracy in experimental results, offering a robust solution for real-time mitigation of SQLi threats and improved web application security.

INTRODUCTION

The increasing reliance on database-driven systems in web-based applications has made them attractive targets for cyberattacks. SQL Injection (SQLi) remains a prevalent attack vector, allowing malicious users to manipulate SQL queries and gain unauthorized access to sensitive data. Traditional SQLi detection methods, including signature-based and rule-based systems, often fail to effectively detect new and sophisticated SQLi attack patterns. Deep learning techniques, particularly Recurrent Neural Networks (RNNs) like Long Short-Term Memory (LSTM) networks, have shown significant promise in addressing these limitations. This project aims to develop a robust SQLi detection and classification system using an LSTM-based deep learning architecture, with a key focus on mitigating overfitting to enhance the model's ability to generalize to unseen data.

The system is trained on a comprehensive dataset of SQL queries, labeled as either benign or SQLi attacks. A two-stage classification strategy is employed: first, to accurately identify malicious queries, and second, to categorize detected attacks into specific SQLi types.

The primary objectives of this project are:

1. Implementing an LSTM-based binary classification model for accurate SQL Injection detection.

2. Developing an LSTM-based multi-class classification model to categorize detected SQL Injection attacks.
3. Employing effective data preprocessing and balancing techniques to optimize model performance.
4. Incorporating regularization techniques to address overfitting and improve the model's generalization capabilities.
5. Thoroughly evaluating the model's performance using relevant metrics, including accuracy, precision, recall, and F1-score.
6. Demonstrating the potential for enhancing the security of database-driven applications through the integration of the developed system.

The proposed LSTM-based approach demonstrates significant potential for accurate and robust SQLi detection, with a reduced risk of overfitting, contributing to the advancement of cybersecurity measures for web applications.

LITERATURE REVIEW

PAPER 1

Title: *Review on Over-fitting and Under-fitting Problems in Machine Learning and Solutions*

Authors: Swathi Pothuganti

Publisher and Year of Publishing: International Journal of Advanced Research in Electrical Electronics and Instrumentation Engineering, 2018

Link: https://www.researchgate.net/publication/344882855_Review_on_over-fitting_and_under-fitting_problems_in_Machine_Learning_and_solutions

Objectives: To analyze and discuss the causes and solutions of overfitting and underfitting in machine learning models.

Methodologies: The paper explores various techniques such as early stopping, network reduction, data expansion, and regularization to mitigate overfitting.

Summary: Overfitting significantly affects machine learning models by reducing their ability to generalize. The study recommends early stopping to prevent models from learning noise, data augmentation to enhance training, and regularization techniques to ensure robust performance.

PAPER 2

Title: *Early Stopping on CNN-LSTM Development to Improve Classification Performance*

Authors: M. K. Anam, S. Defit, H. Haviluddin, L. Efrizoni, M. B. Firdaus

Publisher and Year of Publishing: Journal of Applied Data Sciences, 2024

Link: <https://bright-journal.org/Journal/index.php/JADS/article/view/312/219>

Objectives: To study the impact of early stopping and dropout techniques in CNN-LSTM models for classification tasks.

Methodologies: The study evaluates different CNN-LSTM architectures by applying ReLU, Tanh activation functions, dropout layers, and early stopping mechanisms to prevent overfitting.

Summary: The paper finds that early stopping combined with dropout significantly improves accuracy and generalization, achieving an optimal trade-off between underfitting and overfitting.

PAPER 3

Title: *A Survey on SQL Injection Attacks Detection and Prevention Techniques*

Authors: P. Kumar, R. K. Pateriya

Publisher and Year of Publishing: IEEE International Conference on Computing, Communication and Networking Technologies, 2019

Link: https://www.researchgate.net/publication/261243350_A_survey_on_SQL_injection_attacks_detection_and_prevention_techniques

Objectives: To provide a comprehensive survey of SQL Injection attack types, detection mechanisms, and prevention techniques.

Methodologies: The paper categorizes SQLi detection techniques into black-box testing, white-box testing, machine learning-based approaches, and rule-based filtering systems.

Summary: Machine learning-based detection outperforms traditional rule-based filtering, demonstrating higher accuracy in SQLi prevention while reducing false positives.

PAPER 4

Title: *SQL Injection Attack Detection and Prevention Techniques Using Deep Learning*

Authors: Ding Chen, Qiseng Yan, Chunwang Wu, Jun Zhao

Publisher and Year of Publishing: Journal of Physics: Conference Series, 2021

Link: https://www.researchgate.net/publication/349022673_SQL_Injection_Attack_Detection_and_Prevention_Techniques_Using_Deep_Learning

Objectives: To propose an NLP-based deep learning model for detecting SQL Injection attacks.

Methodologies: The study employs word embeddings, Convolutional Neural Networks (CNN), and Multi-Layer Perceptron (MLP) classifiers to detect SQLi patterns without relying on predefined rule sets.

Summary: The proposed deep learning model significantly reduces false positives and improves SQLi detection rates, making it a promising alternative to traditional security mechanisms.

PAPER 5

Title: *SQL Injection Attack Detection and Prevention Techniques Using Deep Learning*

Authors: Ding Chen, Qiseng Yan, Chunwang Wu, Jun Zhao

Publisher and Year of Publishing: IOP Publishing Ltd, 2021

Link: <https://iopscience.iop.org/article/10.1088/1742-6596/1757/1/012055?utm>

Objectives: To propose an AI-driven SQL Injection detection system without rule-based filtering.

Methodologies: The study uses word embeddings, deep learning architectures, and neural networks to autonomously learn SQLi patterns.

Summary: The paper demonstrates that AI-based approaches enhance SQLi detection accuracy, outperforming conventional signature-based methods while adapting to evolving threats.

PAPER 6

Title: *SQL Injection Detection using Machine Learning: A Review*

Authors: Mohammed A M Oudah, Mohd Fadzli Marhusin

Publisher and Year of Publishing: Malaysian Journal of Science Health & Technology, 2023

Link: <https://mjosht.usim.edu.my/index.php/mjosht/article/view/368?utm>

Objectives: To analyze different machine learning techniques applied for SQL Injection detection.

Methodologies: The paper reviews supervised learning algorithms, including **Decision Trees, Random Forest, Support Vector Machines (SVM), and Neural Networks**, comparing their effectiveness.

Summary: The study highlights the potential of AI-driven SQLi detection, noting that deep learning models achieve higher accuracy while reducing false positives compared to traditional methods.

PAPER 7

Title: *SQL Injection Attack: Detection, Prioritization & Prevention*

Authors: Alan Paul, Vishal Sharma, Oluwafemi Olukoya

Publisher and Year of Publishing: Journal of Information Security and Applications, 2024

Link: <https://www.sciencedirect.com/science/article/pii/S221421262400173X>

Objectives: To propose SQLR34P3R, a holistic system for multi-class SQL Injection detection, prioritization, and prevention.

Methodologies: The system uses a hybrid CNN-LSTM model trained on a dataset of 457,233 samples of web traffic. It integrates risk analysis based on Common Vulnerabilities and Exposures (CVEs) to prioritize security patches.

Summary: The proposed SQLR34P3R system achieved a 97% F1-score in SQL Injection detection, improving real-time threat mitigation by incorporating risk-based prioritization and practical security recommendations for businesses.

PAPER 8

Title: *Advanced LSTM Attention with Hybrid Framework for Detection and Prevention of SQLi and XSS Attacks*

Authors: Mrs. Joshi Padma N., Dr. N. Ravishankar, Dr. M. B. Raju, N. Ch. Ravi

Publisher and Year of Publishing: YMER Journal, 2024

Link: <https://ymerdigital.com/uploads/YMER211166.pdf>

Objectives: To develop an LSTM-Attention-based framework for detecting SQL Injection (SQLi) and Cross-Site Scripting (XSS) attacks.

Methodologies: The study integrates blockchain-based security mechanisms, filtering techniques, and session sanitization with an LSTM attention model for better SQLi and XSS detection.

Summary: The hybrid framework achieves high accuracy, F-score, recall, and precision, demonstrating the effectiveness of deep learning combined with blockchain security for SQLi detection.

PAPER 9

Title: *A Hybrid Approach for Detecting SQL-Injection Using Machine Learning Techniques*

Authors: Hari Krishna, Jared Oluoch, Junghwan Kim

Publisher and Year of Publishing: SCITEPRESS, 2025

Link: <https://www.scitepress.org/Papers/2025/130781/130781.pdf>

Objectives: To improve SQLi detection by combining multiple machine learning algorithms for enhanced accuracy.

Methodologies: A hybrid approach integrates Naïve Bayes, Long Short-Term Memory (LSTM), and Random Forest models, synthesizing outputs for improved classification performance.

Summary: The proposed method demonstrated significantly improved accuracy in detecting SQL Injection attacks, proving that multi-algorithm fusion enhances cybersecurity defenses.

PAPER 10

Title: *Enhancing SQL Injection Prevention: Advanced Machine Learning and LSTM-Based Techniques*

Authors: Ankita Ghosh, Sudip Diyasi, Siddhartha Chatterjee

Publisher and Year of Publishing: International Journal on Computational Modelling Applications, 2024

Link: <https://submissions.adroidjournals.com/index.php/ijcma/article/view/16?utm>

Objectives: To compare machine learning and deep learning models for SQL Injection detection and prevention.

Methodologies: The study evaluates traditional machine learning models vs. Long

Short-Term Memory (LSTM) networks, testing them on SQL Injection datasets.

Summary: Results show that LSTM-based models outperform traditional classifiers, demonstrating higher accuracy and fewer false positives, making them a robust solution for SQLi detection.

PAPER 11

Title: *A Deep Learning Approach Based on Multi-View Consensus for SQL Injection Detection*

Authors: Not specified

Publisher and Year of Publishing: International Journal of Information Security, 2024

Link: <https://link.springer.com/article/10.1007/s10207-023-00791-y>

Objectives: To develop a Bidirectional LSTM-CNN model for multi-view SQL Injection detection.

Methodologies: SQL queries are preprocessed into semantic tags, allowing CNN and LSTM layers to extract different features from multi-view representations.

Summary: The proposed MVC-BiCNN model achieved a 99.96% detection rate, proving that multi-view representations improve SQLi detection accuracy.

PAPER 12

Title: *Evaluating CNN Effectiveness in SQL Injection Attack Detection*

Authors: Muhammad Shahbaz, Gohar Mumtaz, Saleem Zubair, Mudassar Rehman

Publisher and Year of Publishing: Journal of Computing & Biomedical Informatics, 2024

Link: <https://jcbi.org/index.php/Main/article/view/550>

Objectives: To evaluate Convolutional Neural Networks (CNNs) for SQL Injection detection.

Methodologies: The study trains CNN models on a dataset of 109,520 SQL queries and evaluates them using precision, recall, and F1-score.

Summary: The CNN model achieved an accuracy of 97.41%, with 96.50% precision and 99.00% recall, confirming CNN's effectiveness in detecting and classifying SQL Injection attacks.

PAPER 13

Title: *UniEmbed: A Novel Approach to Detect XSS and SQL Injection Attacks Leveraging Multiple Feature Fusion with Machine Learning Techniques*

Authors: Not specified

Publisher and Year of Publishing: Arabian Journal for Science and Engineering, 2025

Link: <https://link.springer.com/article/10.1007/s13369-024-09916-4>

Objectives: To introduce UniEmbed, a machine learning-based detection system for SQL Injection and XSS attacks.

Methodologies: The approach integrates Word2Vec, Universal Sentence Encoder (USE), and FastText embeddings for improved SQLi detection.

Summary: UniEmbed significantly outperformed traditional feature extraction techniques, achieving an F1-score of 99.80% and high accuracy across multiple SQL Injection datasets.

PAPER 14

Title: *Deep Learning Architecture for Detecting SQL Injection Attacks Based on RNN Autoencoder Model*

Authors: Maha Alghawazi, Daniyal Alghazzawi, Suaad Alarifi

Publisher and Year of Publishing: Mathematics Journal, 2023

Link: <https://www.mdpi.com/2227-7390/11/15/3286>

Objectives: To propose an RNN Autoencoder-based model for detecting SQL Injection attacks.

Methodologies: The model is trained on a large SQL Injection dataset and compared with CNN, Decision Tree, Naïve Bayes, SVM, and Random Forest models.

Summary: The RNN Autoencoder achieved 94% accuracy and an F1-score of 92%, proving its effectiveness in detecting SQL Injection attacks while maintaining lower false positives.

PAPER 15

Title: *Detecting SQL Injection Attacks by Binary Gray Wolf Optimizer and Machine Learning Algorithms*

Authors: Bahman Arasteh, Babak Aghaei, Behnoud Farzad, Keyvan Arasteh, Farzad Kiani, Mahsa Torkamanian-Afshar

Publisher and Year of Publishing: Neural Computing and Applications, Springer, 2024

Link: <https://link.springer.com/article/10.1007/s00521-024-09429-z>

Objectives: To enhance SQL injection detection accuracy by using a feature selection method based on the Binary Gray Wolf Optimizer (BGWO) and machine learning classifiers.

Methodologies: The study developed two binary versions of the Gray Wolf Optimizer (GWO) to identify the most important features from a 13-feature SQLi dataset. These optimized features were then used to train various machine learning classifiers, improving detection efficiency while reducing computational costs.

Summary: The proposed model achieved 99.68% accuracy, 99.40% precision, and 98.72% sensitivity, proving that optimized feature selection significantly improves SQLi detection efficiency. The study also demonstrated that selecting only 20% of the most critical features led to increased performance while reducing computational cost.

PAPER 16

Title: *Advanced LSTM Attention with Hybrid Framework for Detection and Prevention of SQLi and XSS Attacks*

Authors: Mrs. Joshi Padma N., Dr. N. Ravishankar, Dr. M. B. Raju, N. Ch. Ravi

Publisher and Year: YMER Journal, 2024

Link: <https://ymerdigital.com/uploads/YMER211166.pdf>

Objectives: To develop an LSTM-Attention-based framework for detecting SQL Injection (SQLi) and Cross-Site Scripting (XSS) attacks.

Methodologies: The study integrates an LSTM model with attention mechanisms, blockchain-based security features, and session sanitization techniques to enhance detection capabilities. The framework focuses on identifying patterns in sequential data while ensuring robust security measures.

Summary: The hybrid framework achieved high accuracy and F1 scores, demonstrating the effectiveness of combining LSTM with attention mechanisms for detecting complex web-based attacks like SQLi and XSS.

These papers provide insights into advanced LSTM-based techniques for SQL Injection detection, highlighting their potential to improve accuracy and adaptability in cybersecurity applications.

RELATED WORK

Numerous studies have focused on developing SQLi detection techniques, employing various approaches, including rule-based, anomaly-based, and machine learning models. Traditional methods, such as signature-based detection, rely on predefined patterns to identify attacks. However, these methods often fail to detect novel SQLi variants due to their limited ability to generalize to unseen attack patterns.

Recent research has explored machine learning models, including Decision Trees, Support Vector Machines (SVMs), and Random Forests, to classify SQL queries. These models offer improved adaptability compared to rule-based systems but often require extensive feature engineering to extract relevant features from SQL queries.

Deep learning-based approaches, particularly Recurrent Neural Networks (RNNs) like Long Short-Term Memory (LSTM) networks, have demonstrated significant promise in SQLi detection. LSTMs are well-suited for processing sequential data like SQL queries, as they can capture long-range dependencies and complex patterns within the query structure. This ability to model sequential information is crucial for SQLi detection, as the order of tokens in a query can drastically alter its meaning and intent, a nuance that simpler models may miss.

Studies have shown the effectiveness of LSTM-based models in SQLi detection. Ghosh et al. (2024) found that LSTM networks outperform traditional classifiers, achieving higher accuracy and fewer false positives. Krishna et al. (2025) further support this, demonstrating that a hybrid approach incorporating LSTM improves detection accuracy. Padma et al. (2024) developed an LSTM-Attention framework, highlighting the benefits of combining LSTM with attention mechanisms for enhanced detection of both SQLi and XSS attacks.

While some research has explored hybrid models, such as the CNN-LSTM architecture used by Paul et al. (2024), our work focuses on a pure LSTM architecture. This choice allows us to specifically investigate the capacity of LSTM networks to effectively capture the sequential nuances of SQL queries for accurate

SQLi detection and fine-grained attack classification. Our approach aims to leverage the inherent strengths of LSTMs in processing sequential data to improve the robustness and accuracy of SQLi detection systems. Furthermore, we address the critical issue of overfitting, a common challenge in deep learning, by incorporating dropout regularization, which helps the model generalize better to unseen data (as suggested by Pothuganti, 2018 and Anam et al., 2024).

METHODOLOGY/ PROPOSED WORK

Our proposed system follows a two-stage LSTM-based deep learning pipeline for SQL Injection (SQLi) detection and classification:

1. Data Preprocessing

To prepare the dataset for model training and ensure balanced learning, we perform the following steps:

- **Dataset Loading and Exploration:**

We load a verified SQL query dataset containing labels (0 for benign queries, 1 for SQLi) and specific SQLi attack types (SQLi_Type column). Exploratory analysis is performed to assess the distribution of attack types.

- **Balancing the Dataset:**

Many SQLi attack types are underrepresented. To address this:

- We fix **500 samples per attack type** by:
 - **Downsampling** overrepresented types.
 - **Synthesizing queries** for underrepresented ones using a custom function that combines fragments of existing queries, generating plausible new samples without using SMOTE (as SMOTE is unsuitable for text).
- Legitimate (non-SQLi) samples are also balanced to match the total number of SQLi samples using the same synthesis method.

- **Tokenization and Padding:**

- Character-level tokenization is applied using Keras' Tokenizer.

- Sequences are padded to a fixed length (`max_seq_length = 100`) to ensure uniform input shape.

2. Model Architecture

We implement two LSTM-based models: a **binary classifier** and a **multi-class classifier**, both using a similar architecture:

- **Embedding Layer:**
Converts character indices into dense vector representations for each query.
- **LSTM Layers:**
Two stacked LSTM layers (with 64 and 32 units respectively) extract sequential features from query patterns, capturing both short and long dependencies.
- **Fully Connected Layers:**
A dense layer with ReLU activation followed by a Dropout layer (rate = 0.3) to prevent overfitting.
- **Output Layer:**
 - For binary classification: a single neuron with sigmoid activation.
 - For multi-class classification: a softmax layer with neurons equal to the number of SQLi attack types.

3. Model Training

- **Stage 1 – Binary Classification:**
The binary model is trained to detect whether a query is an SQLi attack.
- **Stage 2 – Multi-Class Classification:**
Only queries identified as SQLi are used to train the second model, which predicts the specific attack type.
- **Training Enhancements:**
 - **EarlyStopping Callback:**
Training is stopped if the validation loss does not improve for 3 consecutive epochs, preserving the best model weights.

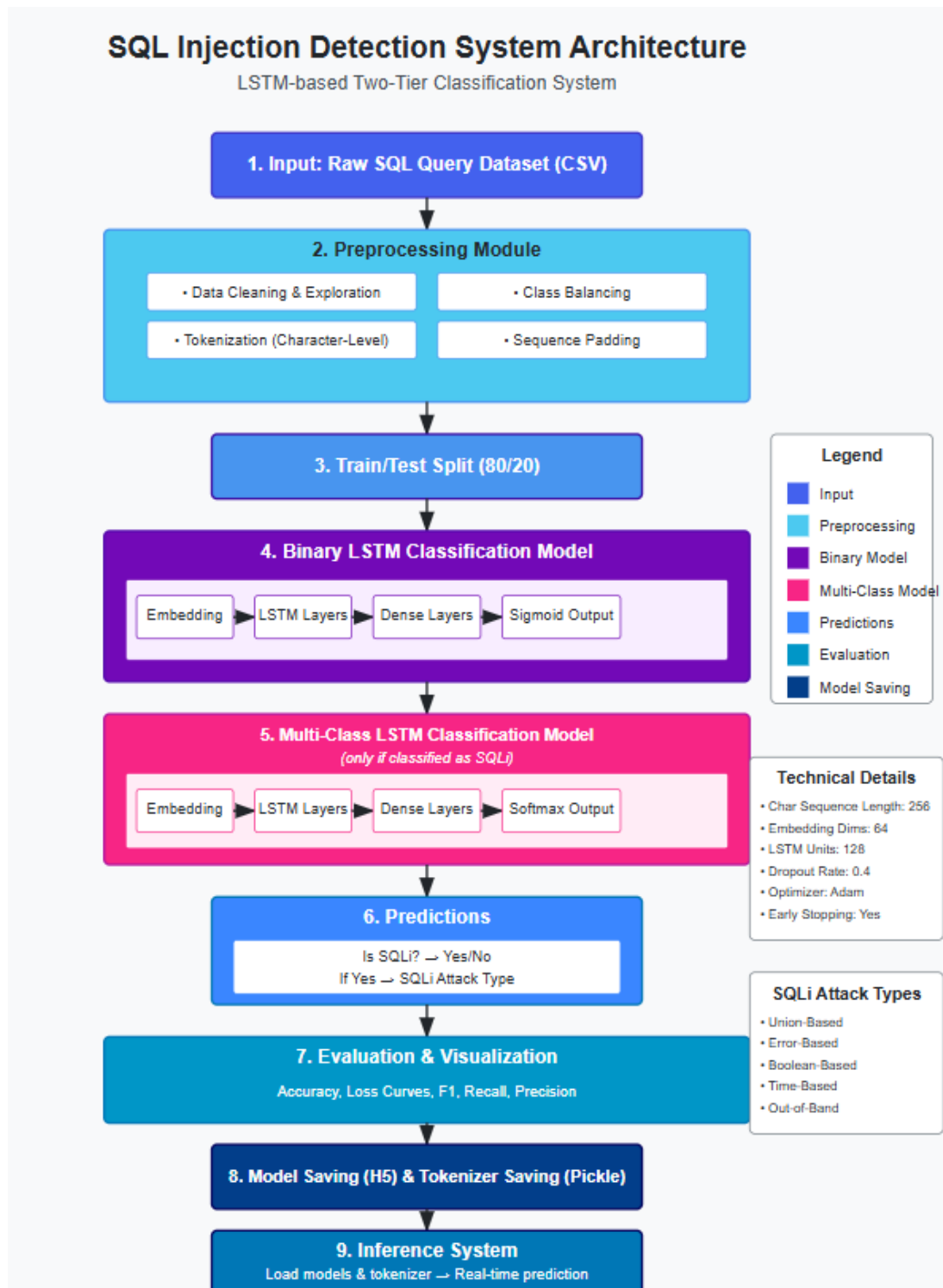
- **Dropout Regularization:**
Dropout is applied in both models to reduce overfitting and improve generalization.
- **Evaluation:**
Both models are evaluated using metrics like accuracy, precision, recall, and F1-score. A training history plot is also generated to visualize model performance over epochs.

4. Inference Pipeline

For real-time prediction:

- A user-input SQL query is preprocessed using the trained tokenizer and padded.
- The binary model first checks for SQLi.
- If detected, the multi-class model predicts the specific type of attack.
- The results are returned with probabilities for both SQLi classification and attack type identification.

DESIGN ARCHITECTURE



ALGORITHM/ PSEUDOCODE

ALGORITHM: SQL Injection Detection System

// Main procedure for SQL injection detection

PROCEDURE DetectSQLInjection(query: String) -> Boolean

// Step 1: Preprocessing

processedQuery = PreprocessQuery(query)

// Step 2: Feature extraction

features = ExtractFeatures(processedQuery)

// Step 3: Classification

injectionProbability = ClassifyQuery(features)

// Step 4: Decision

IF injectionProbability > THRESHOLD THEN

RETURN TRUE // SQL injection detected

ELSE

RETURN FALSE // Legitimate query

END IF

END PROCEDURE

// Preprocessing procedure

PROCEDURE PreprocessQuery(query: String) -> ProcessedQuery

// Convert to lowercase for consistency

normalizedQuery = ToLowerCase(query)

// Tokenize at character level

tokens = TokenizeToCharacters(normalizedQuery)

// Convert characters to numerical indices

indices = MapToIndices(tokens)

// Pad sequence to fixed length

paddedSequence = PadSequence(indices, MAX_SEQUENCE_LENGTH)

```
RETURN paddedSequence
```

```
END PROCEDURE
```

```
// Feature extraction procedure
```

```
PROCEDURE ExtractFeatures(processedQuery: ProcessedQuery) ->  
FeatureVector
```

```
// Apply embedding layer to convert indices to vectors
```

```
embeddings = ApplyEmbedding(processedQuery, EMBEDDING_DIM)
```

```
RETURN embeddings
```

```
END PROCEDURE
```

```
// Classification using LSTM-based deep learning model
```

```
PROCEDURE ClassifyQuery(features: FeatureVector) -> Probability
```

```
// Apply LSTM to capture sequential patterns
```

```
lstmFeatures = ApplyLSTM(features)
```

```
// Apply fully connected layers for classification
```

```
denseFeatures = ApplyDenseLayer(lstmFeatures)
```

```
// Apply dropout for regularization
```

```
regularizedFeatures = ApplyDropout(denseFeatures, DROPOUT_RATE)
```

```
// Final classification
```

```
probability = ApplySigmoid(regularizedFeatures)
```

```
RETURN probability
```

```
END PROCEDURE
```

```
// Model training procedure
```

```
PROCEDURE TrainModel(trainingData: Dataset) -> TrainedModel
```

```
// Split dataset
```

```
trainSet, validationSet = SplitDataset(trainingData)
```



```

// Initialize model with architecture
model = InitializeModel()
// Training loop
FOR epoch = 1 TO MAX_EPOCHS DO
// Process batch
FOR EACH batch IN trainSet DO
features = ExtractFeatures(batch.queries)
predictions = ForwardPass(model, features)
loss = CalculateLoss(predictions, batch.labels)
gradients = CalculateGradients(loss)
UpdateModelParameters(model, gradients)
END FOR
// Validation
validationLoss = EvaluateOnValidation(model, validationSet)

// Early stopping check
IF ShouldEarlyStop(validationLoss) THEN
BREAK
END IF
END FOR
RETURN model
END PROCEDURE

ALGORITHM: SQL Injection Detection System
// Main procedure for SQL injection detection
PROCEDURE DetectSQLInjection(query: String) -> Boolean
// Step 1: Preprocessing
processedQuery = PreprocessQuery(query)
// Step 2: Feature extraction

```

```
features = ExtractFeatures(processedQuery)
// Step 3: Classification
injectionProbability = ClassifyQuery(features)
// Step 4: Decision
IF injectionProbability > THRESHOLD THEN
RETURN TRUE // SQL injection detected
ELSE
RETURN FALSE // Legitimate query
END IF
END PROCEDURE
```

```
// Preprocessing procedure
PROCEDURE PreprocessQuery(query: String) -> ProcessedQuery
// Convert to lowercase for consistency
normalizedQuery = ToLowerCase(query)
// Tokenize at character level
tokens = TokenizeToCharacters(normalizedQuery)
// Convert characters to numerical indices
indices = MapToIndices(tokens)
// Pad sequence to fixed length
paddedSequence = PadSequence(indices, MAX_SEQUENCE_LENGTH)
RETURN paddedSequence
END PROCEDURE
```

```
// Feature extraction procedure
PROCEDURE ExtractFeatures(processedQuery: ProcessedQuery) ->
FeatureVector
// Apply embedding layer to convert indices to vectors
```

```
embeddings = ApplyEmbedding(processedQuery, EMBEDDING_DIM)
RETURN embeddings
END PROCEDURE
```

```
// Classification using LSTM-based deep learning model
PROCEDURE ClassifyQuery(features: FeatureVector) -> Probability
// Apply LSTM to capture sequential patterns
lstmFeatures = ApplyLSTM(features)
// Apply fully connected layers for classification
denseFeatures = ApplyDenseLayer(lstmFeatures)
// Apply dropout for regularization
regularizedFeatures = ApplyDropout(denseFeatures, DROPOUT_RATE)
// Final classification
probability = ApplySigmoid(regularizedFeatures)
RETURN probability
END PROCEDURE
```

```
// Model training procedure
PROCEDURE TrainModel(trainingData: Dataset) -> TrainedModel
// Split dataset
trainSet, validationSet = SplitDataset(trainingData)
// Initialize model with architecture
model = InitializeModel()
// Training loop
FOR epoch = 1 TO MAX_EPOCHS DO
// Process batch
FOR EACH batch IN trainSet DO
features = ExtractFeatures(batch.queries)
```

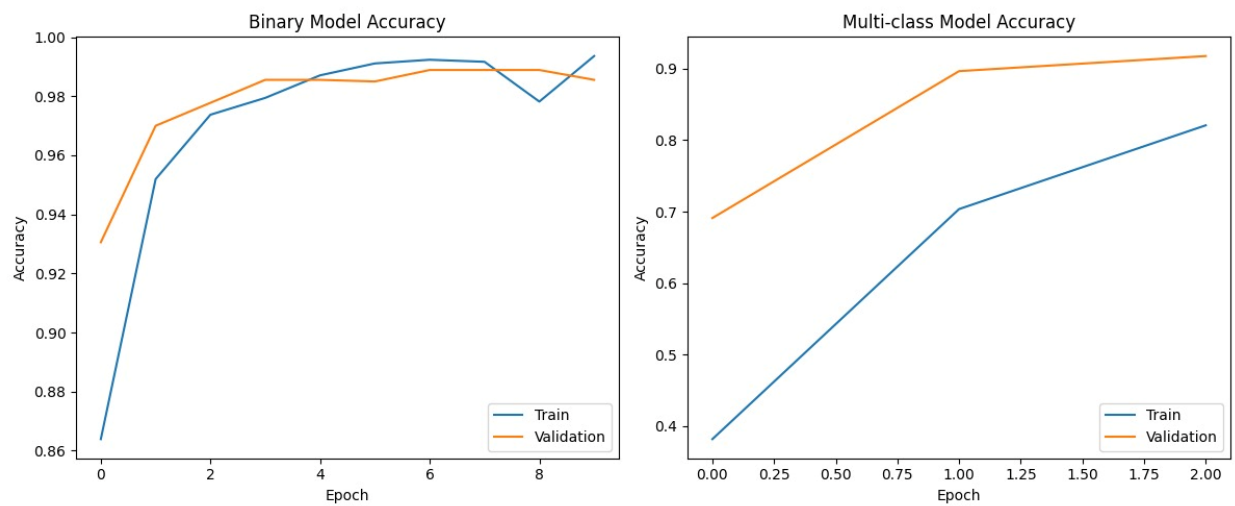
```

predictions = ForwardPass(model, features)
loss = CalculateLoss(predictions, batch.labels)
gradients = CalculateGradients(loss)
UpdateModelParameters(model, gradients)
END FOR
// Validation
validationLoss = EvaluateOnValidation(model, validationSet)

// Early stopping check
IF ShouldEarlyStop(validationLoss) THEN
BREAK
END IF
END FOR
RETURN model
END PROCEDURE

```

TRAINING HISTORY



OUTPUT

Output for preprocessing and training

```
Loading dataset from /content/Verified_SQL_Dataset_with_Types.csv...
Original dataset shape: (30919, 3)
Label distribution:
Label
0      19537
1      11382
Name: count, dtype: int64
SQLi Type distribution:
SQLi_Type
0      19537
1       7015
9       4086
4        160
2         102
8           8
6           4
7           3
3           2
5           2
Name: count, dtype: int64
Attack type 7 has 3 samples. Generating 497 more.
Attack type 3 has 2 samples. Generating 498 more.
Attack type 6 has 4 samples. Generating 496 more.
Attack type 4 has 160 samples. Generating 340 more.
Attack type 8 has 8 samples. Generating 492 more.
Attack type 2 has 102 samples. Generating 398 more.
Attack type 5 has 2 samples. Generating 498 more.
Total attack samples after balancing: 4500

Balanced dataset shape: (9000, 3)
Label distribution:
Label
1      4500
0      4500
Name: count, dtype: int64
SQLi Type distribution:
SQLi_Type
0      4500
1       500
7       500
3       500
6       500
9       500
4       500
8       500
2       500
5       500
Name: count, dtype: int64
Binary model summary:
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/embedding.py:90: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
conv1d_1 (Conv1D)	?	0 (unbuilt)
max_pooling1d_1 (MaxPooling1D)	?	0
lstm_1 (LSTM)	?	0 (unbuilt)
lstm_2 (LSTM)	?	0 (unbuilt)
dense_1 (Dense)	?	0 (unbuilt)
dropout_1 (Dropout)	?	0
dense_2 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Multi-class model summary:

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
conv1d_2 (Conv1D)	?	0 (unbuilt)
max_pooling1d_2 (MaxPooling1D)	?	0
lstm_3 (LSTM)	?	0 (unbuilt)
lstm_4 (LSTM)	?	0 (unbuilt)
dense_3 (Dense)	?	0 (unbuilt)
dropout_2 (Dropout)	?	0
dense_4 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Training binary classification model...

Epoch 1/10

225/225 ————— 27s 70ms/step - accuracy: 0.7607 -
loss: 0.4720 - val_accuracy: 0.9556 - val_loss: 0.1306

Epoch 2/10

```
225/225 ----- 15s 65ms/step - accuracy: 0.9744 -  
loss: 0.1074 - val_accuracy: 0.9850 - val_loss: 0.0599  
Epoch 3/10  
225/225 ----- 20s 62ms/step - accuracy: 0.9852 -  
loss: 0.0620 - val_accuracy: 0.9856 - val_loss: 0.0627  
Epoch 4/10  
225/225 ----- 14s 63ms/step - accuracy: 0.9850 -  
loss: 0.0572 - val_accuracy: 0.9872 - val_loss: 0.0397  
Epoch 5/10  
225/225 ----- 21s 67ms/step - accuracy: 0.9887 -  
loss: 0.0410 - val_accuracy: 0.9883 - val_loss: 0.0289  
Epoch 6/10  
225/225 ----- 15s 65ms/step - accuracy: 0.9935 -  
loss: 0.0308 - val_accuracy: 0.9878 - val_loss: 0.0531  
Epoch 7/10  
225/225 ----- 15s 66ms/step - accuracy: 0.9946 -  
loss: 0.0225 - val_accuracy: 0.9961 - val_loss: 0.0145  
Epoch 8/10  
225/225 ----- 14s 63ms/step - accuracy: 0.9973 -  
loss: 0.0123 - val_accuracy: 0.9983 - val_loss: 0.0071  
Epoch 9/10  
225/225 ----- 21s 66ms/step - accuracy: 0.9985 -  
loss: 0.0083 - val_accuracy: 0.9978 - val_loss: 0.0098  
Epoch 10/10  
225/225 ----- 20s 62ms/step - accuracy: 0.9970 -  
loss: 0.0117 - val_accuracy: 0.9956 - val_loss: 0.0196  
  
Training multi-class classification model...  
Epoch 1/10  
113/113 ----- 15s 84ms/step - accuracy: 0.1832 -  
loss: 2.1506 - val_accuracy: 0.6178 - val_loss: 1.1061  
Epoch 2/10  
113/113 ----- 9s 77ms/step - accuracy: 0.6519 -  
loss: 1.0394 - val_accuracy: 0.8111 - val_loss: 0.4857  
Epoch 3/10  
113/113 ----- 10s 72ms/step - accuracy: 0.7868 -  
loss: 0.6055 - val_accuracy: 0.9133 - val_loss: 0.3208  
Epoch 4/10  
113/113 ----- 8s 72ms/step - accuracy: 0.8482 -  
loss: 0.4616 - val_accuracy: 0.9244 - val_loss: 0.2430  
Epoch 5/10  
113/113 ----- 11s 75ms/step - accuracy: 0.8761 -  
loss: 0.3807 - val_accuracy: 0.9544 - val_loss: 0.1725  
Epoch 6/10  
113/113 ----- 9s 78ms/step - accuracy: 0.8910 -  
loss: 0.3208 - val_accuracy: 0.9556 - val_loss: 0.1447  
Epoch 7/10  
113/113 ----- 7s 65ms/step - accuracy: 0.9152 -  
loss: 0.2690 - val_accuracy: 0.9611 - val_loss: 0.1275  
Epoch 8/10  
113/113 ----- 11s 69ms/step - accuracy: 0.9150 -  
loss: 0.2483 - val_accuracy: 0.9556 - val_loss: 0.1436  
Epoch 9/10
```

```

113/113 ----- 9s 78ms/step - accuracy: 0.9102 -
loss: 0.2801 - val_accuracy: 0.9567 - val_loss: 0.1263
Epoch 10/10
113/113 ----- 9s 77ms/step - accuracy: 0.9270 -
loss: 0.2229 - val_accuracy: 0.9700 - val_loss: 0.0934
57/57 ----- 1s 20ms/step - accuracy: 0.9977 -
loss: 0.0104

Binary model - Test accuracy: 0.9983
29/29 ----- 1s 20ms/step - accuracy: 0.9716 -
loss: 0.0937
Multi-class model - Test accuracy: 0.9700
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.

Models and tokenizer saved successfully.

Example predictions:
1/1 ----- 0s 412ms/step

Query: SELECT * FROM users
Is SQLi: False (Probability: 0.0022)
1/1 ----- 0s 49ms/step

Query: SELECT * FROM users WHERE username='admin' OR 1=1
Is SQLi: False (Probability: 0.0006)
1/1 ----- 0s 51ms/step
1/1 ----- 0s 420ms/step

Query: 1'; DROP TABLE users; --
Is SQLi: True (Probability: 1.0000)
Attack Type: Error-based (Probability: 0.5856)
1/1 ----- 0s 53ms/step

Query: SELECT * FROM products WHERE category='Electronics'
Is SQLi: False (Probability: 0.0000)

```

Output for prediction:

```

Enter an SQL query to analyze: aadarsh pranav sureshbabu
1/1 ----- 0s 357ms/step
Query: aadarsh pranav sureshbabu
SQLi Probability: 0.0001
Is SQLi: No

Prediction Result: {'query': 'aadarsh pranav sureshbabu', 'is_sqli': False, 'sqli_probability':
5.6176450016209856e-05, 'attack_type': 'Not SQLi', 'attack_type_probability': 0.0}

```


CONCLUSION

In this project, we developed an LSTM-based deep learning system for intelligent detection and classification of SQL Injection (SQLi) attacks. The system uses a two-stage classification pipeline: a binary classifier to distinguish between benign and malicious SQL queries, and a multi-class classifier to categorize detected SQLi attacks into specific types.

To ensure balanced and representative training data, we applied synthetic text generation techniques to augment underrepresented SQLi classes and legitimate queries. Character-level tokenization and sequence padding were used to capture fine-grained patterns in SQL syntax, enhancing the model's ability to learn subtle injection indicators.

The binary classifier achieved nearly perfect accuracy, effectively identifying malicious queries with strong generalization. The multi-class model, while more complex, showed consistent learning and accurately distinguished between various SQLi attack types. Regularization techniques like dropout and early stopping helped minimize overfitting during training.

Overall, the proposed approach demonstrates that LSTM networks are highly effective in detecting and classifying SQLi threats. This system addresses the limitations of traditional rule-based methods by learning directly from query structures and adapting to evolving attack patterns. Future work may explore architectural enhancements, real-time deployment, and integration with web application security tools for proactive defense against SQL injection vulnerabilities.